

Manual Técnico — EcoCiclos: Guardianes de la Materia

1. Stack

Capa	Tecnología
Lenguaje	Kotlin
UI	Jetpack Compose + Material 3
Navegación	Navigation Compose
Persistencia	Room (SQLite)
Concurrencia/estado	Coroutines + Flow/StateFlow
Build	Gradle Kotlin DSL + Gradle Wrapper, JDK 17
minSdk	24 · <code>targetSdk/compileSdk</code>
Tests	JUnit4 + Truth + Robolectric + Room Testing + Coroutines Test

Todas las versiones de dependencias están **fijadas** (no dinámicas), en `app/build.gradle.kts`.

2. Arquitectura: MVVM + Clean Architecture

data/	
local/	Room: entities, DAOs, AppDatabase, seed, DataStore (PreferenciasApp)
repository/	EcoRepositoryImpl (Room -> dominio) + Mappers
domain/	
model/	Modelos puros de Kotlin (sin Android, sin Room)
repository/	EcoRepository (interfaz – el "puerto" de dominio)
usecase/	Toda la lógica de negocio, 100% testeable sin UI
ui/	
<feature>/	Screen (Composable) + ViewModel por feature
components/	Composables reutilizables (ilustraciones, tarjetas, barras)
theme/	Color, tipografía, Theme
navigation/	Routes + EcoNavGraph

Regla dura seguida en todo el proyecto: **ninguna Composable ni ViewModel llama a Room directamente**. Todo pasa por `EcoRepository` (interfaz en `domain/repository`), implementado por `EcoRepositoryImpl` en `data/repository`. Los use cases (`domain/usecase/*`) no importan nada de

Android ni de Room: reciben modelos de dominio y devuelven modelos de dominio, por lo que corren como tests JVM puros sin Robolectric.

3. Flujo de datos

`Room (Flow)` → `EcoRepositoryImpl` (combina flows + aplica use cases de progreso) → `EcoRepository` (interfaz) → `ViewModel (StateFlow` expuesto a la UI) → `Composable (collectAsState)`.

Ejemplo real: `GlobeViewModel` combina `repository.observeBiomass()` y `repository.observePerfil()` con `combine(...)`, aplica `CalcularProgresoGlobalUseCase`, y expone un único `StateFlow<GlobeUiState>` con `stateIn(viewModelScope, WhileSubscribed(5000), ...)`.

4. Inyección de dependencias

El proyecto **no usa Hilt/Koin** deliberadamente: su tamaño no lo justifica. `EcoCiclosApp` (la `Application`) construye `AppDatabase`, `EcoRepositoryImpl` y los use cases una sola vez en `onCreate()`. `ui/ViewModelFactory.kt` expone `crearViewModel { app -> MiViewModel.crear(app, ...) }`, un helper `Compose` que resuelve el `Application` actual vía `LocalContext` y crea el `ViewModel` con una `ViewModelProvider.Factory` mínima.

5. Los tres motores de puzzle (dominio)

- `ValidarConexionRestauradorUseCase` — valida el mapa `claveOrigen -> claveDestinoElegida` contra `ElementoNivel.destinoCorrectoClave`.
- `ValidarRutaEnrutadorUseCase` — valida `claveFuente -> claveDestinoElegida`, comprobando **tanto** que la clave de destino sea la correcta **como** que el `TipoRecurso` de la fuente coincida con el que acepta el destino (así se modela la "contaminación": tipo incorrecto aunque el niño haya tocado un nodo real).
- `EvaluarLaboratorioUseCase` — calcula `magnitudEfecto` (0..1) y `SeveridadLab` (NORMAL/ALERTA/CRITICO) a partir de la distancia del valor actual al `umbralCritico`, sin ningún texto "quemado" fuera de los mensajes configurados por nivel.
- `calcularEstrellas()` (en `ValidarConexionRestauradorUseCase.kt`, `internal`) centraliza la regla de puntuación: 3 estrellas sin errores, 2 con un error, 1 con más, 0 si no se completó.

Estos tres use cases devuelven siempre un `ResultadoNivel`, que `CompletarNivelUseCase` recibe y orquesta: persiste el intento (`registrarResultadoNivel`, siempre, incluso en fallo), y si `completado == true`, intenta desbloquear la carta asociada, recalcula el % del bioma, y evalúa insignias nuevas vía `DesbloquearInsigniaUseCase`.

6. Progresión y desbloqueos

- `CalcularEstadoNivelesUseCase`: el primer nivel de un bioma siempre está `DISPONIBLE`; el resto se desbloquea solo cuando el anterior tiene `completado = true`. Con 3 estrellas, el estado sube a `DOMINADO`.
- `CalcularEstadoBiomasUseCase`: el primer bioma siempre está `DISPONIBLE/INICIADO`; los siguientes permanecen `BLOQUEADO` hasta que el bioma anterior llega a 100% (`COMPLETADO`).
- `CalcularProgresoBiomaUseCase` / `CalcularProgresoGlobalUseCase`: porcentajes calculados en tiempo real a partir de `progreso_nivel`, nunca almacenados como un número fijo.

7. Los tres mecanismos de interacción (UI)

- `RestauradorCiclo.kt` — Drag & drop real con `Modifier.pointerInput { detectDragGestures(...) }` sobre cada elemento de origen, `graphicsLayer { translationX/Y }` para el arrastre visual, y detección de colisión contra los `Rect` de los destinos (capturados con `onGloballyPositioned + positionInRoot()`).
- `EnrutadorRecursos.kt` — Mismo patrón de arrastre, pero dibuja la trayectoria en tiempo real sobre un `Canvas` (líneas confirmadas + la línea activa mientras el dedo se mueve).
- `LaboratorioReacciones.kt` — `Slider` de Material3 ligado a `EvaluarLaboratorioUseCase`; el `Canvas` de la visualización anima con `animateFloatAsState` la "barra de salud del ecosistema" en función de `magnitudEfecto`.

8. Persistencia

Ver `BASE_DE_DATOS.md` para el esquema completo. Resumen: 11 entidades Room, todas con claves foráneas `CASCADE` donde corresponde, sin ninguna tabla "en memoria" sustituyendo a Room. `DatabaseSeeder` puebla la base de datos una única vez (comprueba `biomaDao().contar() > 0` antes de insertar), por lo que reabrir la app no duplica datos.

9. Cómo añadir un bioma o nivel nuevo

1. Añade la fila de `BiomaEntity` (o `NivelEntity`) en `SeedData.kt`.
2. Si es `RESTAURADOR`, añade sus `ElementoNivelEntity` (pares origen/destino). Si es `ENRUTADOR`, sus `NodoRutaEntity` (fuentes + destinos con `TipoRecurso`). Si es `LABORATORIO`, su única `VariableLabEntity`.
3. Opcionalmente, una `CartaEntity` con `nivelDesbloqueoId` apuntando al nuevo nivel.
4. No se requiere ningún cambio de código: `SimuladorScreen` ya despacha por `TipoReto` a la mecánica correspondiente, y las pantallas de bioma/globo leen todo desde Room.

10. Pruebas

76 tests JVM (`./gradlew testDebugUnitTest`), sin necesitar emulador: - `domain/*Test.kt` — use cases puros, con `FakeEcoRepository` para `CompletarNivelUseCase`. - `data/*Test.kt` — Room en

memoria vía Robolectric (`RoomTestBase`), cubriendo cada DAO, el `DatabaseSeeder` (incluyendo idempotencia) y `EcoRepositoryImpl` de punta a punta.

11. Compilación

Ver `BUILD_REPORT.md`: no verificada localmente por falta de SDK de Android/acceso de red en el entorno de generación; el workflow de CI (`.github/workflows/android.yml`) es el que compila realmente.